

项目申请书

项目名称 : Asterinas 可靠 OverlayFS 支持

项目主导师 : 陈澄钧 chenchengjun.ccj@antgroup.com

申请人 : 安昱达

日期 : 2026.8.21

邮箱 : anyuda01@163.com

目录

- 项目申请书
 - 目录
 - 一、项目背景与目标
 - 二、项目详细方案
 - 1. 架构重构
 - 2. 挂载语义与层管理
 - 3. 并发、可靠性与权限
 - 4. 操作语义补齐
 - 5. 测试与交付
 - 三、项目开发时间计划 (3个月)
 - 第 1 个月: 基础与基线
 - 第 2 个月: 功能与可靠性
 - 第 3 个月: 验证与交付
 - 四、预期成果

一、项目背景与目标

Asterinas 是一个使用 Rust 编写、兼容 Linux ABI 的通用操作系统内核，采用 Framekernel 架构，安全内核代码中禁止使用 unsafe Rust。OverlayFS 是 Linux 容器栈中常用的联合文件系统，它将一个或多个只读 lower layer 与可写 upper layer 合并为统一视图，对容器镜像分层和容器可写层至关重要。

Asterinas 目前已有 OverlayFS 原型，支持基本挂载、多 lower layer、目录合并、copy-up、whiteout、opaque directory 和部分 xattr 操作。但现有实现仍存在 rename 未实现、并发操作可能竞态、whiteout 与 workdir 行为未完全对齐 Linux、同步与异常路径不完整、测试覆盖不足等问题，尚不能作为可靠的容器可写层使用。

本项目以 Linux 用户可见语义为基准，目标是完善 Asterinas OverlayFS 的核心功能与可靠性，建立可重复运行的 xfstests 验证体系，并完成容器场景基本验收，使 OverlayFS 能够可靠地支撑容器镜像分层与可写层。

二、项目详细方案

1. 架构重构

原始 OverlayFS 是单文件实现，挂载、查找、copy-up、目录操作、xattr 等逻辑耦合在一起，代码规模增长后难以定位问题，也不利于社区并行开发和后续扩展。项目将按职责将其重构为模块化实现，明确各模块边界和数据流，同时保留清晰的锁与状态不变量。这样既能提升可维护性，也为后续功能迭代、评审和长期维护提供更好的工程基础。

2. 挂载语义与层管理

旧实现只做简单的挂载选项解析，对只读挂载、upper/workdir 约束和多层校验支持不足。挂载是 OverlayFS 正确性的基础，项目将完善挂载选项的解析与校验，包括 lowerdir、upperdir、workdir 的格式和组合规则，支持无 upperdir 的只读挂载，并拒绝未知选项和非法组合。对于可写挂载，将校验 upper 与 workdir 位于同一文件系统、upper 可写、workdir 不与 upper 重叠，并建立 upper/workdir 的独占机制，避免多个挂载共享同一可写层造成状态混乱。同时完善多 lower layer 的组装与重叠检查，探测底层文件系统对 xattr、d_type、mknod 等能力的支持，确保挂载后的行为与 Linux 语义一致。

3. 并发、可靠性与权限

旧实现中多处 TODO 已标明存在竞态风险，异常与同步处理也不完整。本项目将**先确定 OverlayFS 的锁序与状态不变量**，作为所有并发路径的统一约束；**再在此约束下**完善多目录操作时的加锁顺序、copy-up 的并发协调，以及操作失败后的回滚与清理、fsync 同步保证；**最后**补齐权限与安全边界，确保必选范围内无已知 panic、死锁或数据错误。

4. 操作语义补齐

旧实现中 rename 未实现，whiteout 采用非 Linux 风格，部分操作和错误码也不完整。在可靠性框架之上，逐项对齐 Linux 用户可见行为。目录操作方面，实现和完善 rename，覆盖同目录与跨目录场景，处理 whiteout 目标、替换目标、目录非空等边界；完善 unlink/rmdir/link，确保删除 lower 文件时生成正确的 whiteout，目录删除时处理 opaque 和隐藏子项。copy-up 方面，完善触发条件、父目录递归、workdir 暂存和原子发布，保证元数据、数据与 xattr 正确拷贝。whiteout 与 workdir 行为将按 Linux 语义对齐，包括 whiteout 表示形式、识别规则和工作目录清理。xattr 方面，完善 overlay 私有标记与用户可见 xattr 的区分，并修正错误码与同步行为，覆盖目录合并、删除隐藏、跨层访问等典型场景。

5. 测试与交付

项目将复用社区已有的 xfstests 多文件系统框架 (PR #3298)，为 OverlayFS 建立专用配置、用例列表、运行脚本和必要的测试镜像生成逻辑；在当前 Asterinas 底座支持的范围内运行 overlay 相关用例，建立 PASS/FAIL/NOTRUN 基线，并针对已支持场景逐步补充测试覆盖。对于暂时无法运行的用例，记录为能力限制并在文档中说明原因。在社区协助下，使用成果内核完成 Kata 或 CoCo 常规模式的 OverlayFS 基本文件操作验收，保留可核验日志。最后输出设计文档，说明锁与状态不变量、与 Linux 的语义差异、未支持特性和已知限制，并按 Asterinas 贡献规范以可评审的 PR 提交。

三、项目开发时间计划（3个月）

第 1 个月：基础与基线

- 完成 OverlayFS 架构重构与模块划分；
- 完善挂载语义与层管理；
- 搭建 OverlayFS xfstests 测试配置与运行基线；
- 梳理与 Linux 的语义差异，形成问题清单和锁/状态设计初稿，作为后续实现的依据。

第 2 个月：功能与可靠性

- 在可靠性框架下实现和完善 rename、目录操作、copy-up、whiteout/workdir、xattr 等语义；
- 补齐并发控制、异常路径与权限检查；
- 随功能进展补充 xfstests 测试用例，对已支持场景持续验证并修复问题，确保不引入新的 panic、死锁或数据错误。

第 3 个月：验证与交付

- 全量运行可支持的 xfstests 用例，分析 PASS/FAIL/NOTRUN 结果并修复剩余问题；
- 在社区协助下完成 Kata/CoCo 容器场景基本文件操作验收并保留日志；
- 编写技术文档，整理增量 PR，完成格式化、编译、lint 与合入前检查，推动成果合入 Asterinas。

整个周期以小步验证和持续集成为原则，每阶段结束都有可运行结果和测试证据，确保最终成果可复现、可评审。

四、预期成果

以上成果均以可复现结果为准，避免仅提供演示或一次性运行结果。

1. 可合入 Asterinas 的 OverlayFS 功能与可靠性改进，必选范围内无已知 panic、死锁或数据错误；
2. 可复现的 OverlayFS xfstests 配置、用例列表、运行脚本和测试镜像生成逻辑，支持持续测试；
3. 技术文档：实现设计、锁与状态不变量、与 Linux 的语义差异、未支持特性和已知限制；
4. 遵循 Asterinas 贡献规范、便于社区评审的 PR，并完成相关格式化、编译与 lint 检查。